

Risk-Triggered Adaptive MPC Safety Filtering with LQR Nominal Control for Differential-Drive Robot Navigation

Abstract—This paper proposes a risk-triggered adaptive predictive safety-filtering architecture for obstacle-aware navigation of differential-drive robots. Instead of directly switching or blending Linear Quadratic Regulator (LQR) and Model Predictive Control (MPC) commands, the proposed method uses LQR as a low-cost nominal tracking controller and activates an adaptive nonlinear MPC safety filter only when the predicted LQR rollout violates obstacle safety constraints. Online Least Mean Squares (LMS) adaptation estimates actuation mismatch in the linear and angular velocity channels, allowing the safety filter to predict robot motion under model uncertainty. The adaptive MPC minimally modifies the LQR command while enforcing obstacle clearance and actuator constraints. A fallback safety layer is used when the MPC becomes infeasible due to sudden obstacle appearance, model-estimate changes, or unavoidable constraint violations. The proposed method is evaluated in ROS2/Gazebo with injected actuation mismatch, sensor noise, static and dynamic obstacles, and compared against LQR-only, MPC-only, adaptive MPC-only, and blended LQR-MPC baselines.

Index Terms—Adaptive MPC, LQR, safety filter, mobile robot navigation, obstacle avoidance, LMS adaptation, Gazebo, differential-drive robot.

I. INTRODUCTION

Autonomous mobile robots require accurate trajectory tracking and reliable obstacle avoidance under actuator uncertainty, localization error, and dynamic environmental conditions. LQR provides an efficient and stabilizing feedback law for trajectory tracking, but it does not naturally handle state and obstacle constraints. MPC can explicitly handle constraints, but solving MPC continuously may be computationally expensive for real-time robotic platforms.

A naive hybrid strategy that switches or blends LQR and MPC commands can lead to unsafe behavior. In particular, even if the MPC command is safe, a convex blend between a safe MPC command and an unsafe LQR command is not guaranteed to preserve obstacle avoidance. Therefore, this work proposes a different formulation: LQR is treated as the nominal tracking controller, while adaptive MPC is used as a predictive safety filter.

II. MAIN CONTRIBUTIONS

The main contributions of this paper are:

- 1) A risk-triggered controller architecture where LQR is used for nominal tracking and adaptive MPC is activated only when the predicted LQR rollout becomes unsafe.
- 2) An LMS-based online parameter adaptation law for estimating linear and angular velocity actuation mismatch.

- 3) A least-invasive adaptive MPC safety filter that minimizes deviation from the LQR command while enforcing obstacle avoidance.
- 4) A fallback safety layer for infeasible MPC cases caused by sudden obstacles, parameter jumps, or unavoidable safety constraint violations.
- 5) A ROS2/Gazebo evaluation with injected actuation mismatch, odometry or LiDAR noise, dynamic obstacles, solver-time profiling, intervention maps, and jerk-based smoothness metrics.

III. SYSTEM MODEL

The differential-drive robot state is

$$x = [p_x \ p_y \ \theta]^T, \quad (1)$$

where p_x, p_y are planar coordinates and θ is the yaw angle. The control input is

$$u = [v \ \omega]^T, \quad (2)$$

where v is the commanded linear velocity and ω is the commanded angular velocity.

To model actuation mismatch, two unknown scaling parameters are introduced:

$$\vartheta = [\vartheta_v \ \vartheta_\omega]^T. \quad (3)$$

The adapted kinematic model is

$$\dot{p}_x = \vartheta_v v \cos \theta, \quad (4)$$

$$\dot{p}_y = \vartheta_v v \sin \theta, \quad (5)$$

$$\dot{\theta} = \vartheta_\omega \omega. \quad (6)$$

Using sampling time Δt , the discrete-time model is written as

$$x_{k+1} = f(x_k, u_k, \hat{\vartheta}_k), \quad (7)$$

where $\hat{\vartheta}_k$ is the current online estimate.

IV. LMS PARAMETER ADAPTATION

The LMS estimator updates the actuation scale parameters using one-step prediction error. Let

$$e_k = x_k - \hat{x}_k \quad (8)$$

be the difference between the measured state and the predicted state. The parameter sensitivity matrix is

$$\Phi_k = \Delta t \begin{bmatrix} v_k \cos \theta_k & 0 \\ v_k \sin \theta_k & 0 \\ 0 & \omega_k \end{bmatrix}. \quad (9)$$

The LMS update law is

$$\hat{\vartheta}_{k+1} = \Pi_{\Theta} \left(\hat{\vartheta}_k + \Gamma \Phi_k^T e_k \right), \quad (10)$$

where Γ is the adaptation gain and Π_{Θ} projects the estimate into bounded admissible limits.

A. Persistent Excitation Limitation

The LMS estimator requires sufficient excitation in the corresponding input channel. For example, if the robot moves in a straight line for a long duration, then $\omega_k \approx 0$ and the estimator cannot reliably learn the angular velocity scale ϑ_{ω} . To avoid meaningless parameter drift, the update for each parameter is paused when its corresponding regressor magnitude is below a threshold:

$$\text{update } \hat{\vartheta}_v \text{ only if } |v_k| > \epsilon_v, \quad (11)$$

$$\text{update } \hat{\vartheta}_{\omega} \text{ only if } |\omega_k| > \epsilon_{\omega}. \quad (12)$$

This prevents the estimator from adapting parameters when the measured data does not contain enough information. In the experiments, curved trajectories and obstacle-avoidance maneuvers provide excitation for both velocity channels.

V. NOMINAL LQR CONTROLLER

The LQR controller is designed around a locally linearized model:

$$\tilde{x}_{k+1} = A_k \tilde{x}_k + B_k \tilde{u}_k. \quad (13)$$

The discrete-time LQR cost is

$$J_{LQR} = \sum_{k=0}^{\infty} (\tilde{x}_k^T Q \tilde{x}_k + \tilde{u}_k^T R \tilde{u}_k). \quad (14)$$

The feedback gain is

$$K_k = (R + B_k^T P B_k)^{-1} B_k^T P A_k, \quad (15)$$

where P is obtained from the discrete algebraic Riccati equation. The nominal command is

$$u_{LQR,k} = u_{ref,k} - K_k(x_k - x_{ref,k}). \quad (16)$$

VI. RISK-TRIGGERED SAFETY FILTER

Before applying $u_{LQR,k}$, the controller predicts the future LQR rollout over a short horizon:

$$\hat{x}_{i+1} = f(\hat{x}_i, u_{LQR,i}, \hat{\vartheta}_k). \quad (17)$$

For obstacle j with center $p_{obs,j}$ and radius $r_{obs,j}$, define

$$h_j(x) = \|p - p_{obs,j}\|_2 - (r_{obs,j} + d_{safe}), \quad (18)$$

where $p = [p_x, p_y]^T$.

The predicted LQR rollout is considered safe if

$$h_j(\hat{x}_i) \geq 0, \quad \forall i \in \{0, \dots, N\}, \forall j. \quad (19)$$

If the rollout is safe, the LQR command is applied directly. Otherwise, the adaptive MPC safety filter is activated.

VII. ADAPTIVE MPC SAFETY FILTER

The adaptive MPC safety filter solves:

$$\min_{\mathbf{x}, \mathbf{u}} \sum_{i=0}^{N-1} (\|x_i - x_{ref,i}\|_Q^2 + \|u_i - u_{LQR,i}\|_R^2 + \|\Delta u_i\|_S^2) + \|x_N - x_{ref,N}\|_P^2 \quad (20)$$

subject to

$$x_0 = x_k, \quad (21)$$

$$x_{i+1} = f(x_i, u_i, \hat{\vartheta}_k), \quad (22)$$

$$u_{min} \leq u_i \leq u_{max}, \quad (23)$$

$$\|p_i - p_{obs,j}\|_2 \geq r_{obs,j} + d_{safe}, \quad \forall i, j. \quad (24)$$

The key term is

$$\|u_i - u_{LQR,i}\|_R^2, \quad (25)$$

which makes the MPC a least-invasive safety filter instead of a replacement controller.

A. Recursive Feasibility Discussion

The MPC safety filter may become infeasible when a sudden obstacle appears too close to the robot, when the LMS-estimated dynamics change abruptly, or when actuator limits make it impossible to satisfy obstacle constraints within the prediction horizon. Therefore, recursive feasibility is not assumed unconditionally. Instead, the controller includes an explicit backup safety layer that activates when the MPC solver reports infeasibility or exceeds a real-time computation limit.

VIII. BACKUP SAFETY LAYER

If adaptive MPC is infeasible or too slow, a backup controller is used. A simple emergency policy is

$$v = 0, \quad \omega = k_{\omega} \cdot \text{sign}(\theta_{away} - \theta), \quad (26)$$

where θ_{away} points away from the nearest obstacle.

A stronger backup option is a control barrier function quadratic program:

$$\min_u \|u - u_{LQR}\|^2 \quad (27)$$

$$\text{s.t. } \dot{h}_j(x, u) + \alpha h_j(x) \geq 0, \quad \forall j, \quad (28)$$

$$u_{min} \leq u \leq u_{max}. \quad (29)$$

IX. BASELINE CONTROLLERS

The proposed method is compared against four baselines.

A. LQR Only

The robot applies u_{LQR} directly without obstacle-aware correction.

B. MPC Only

The robot continuously solves MPC at every control step.

C. Adaptive MPC Only

The robot continuously solves adaptive MPC using the LMS-updated model.

D. Blended LQR-MPC

The blended baseline is defined explicitly as

$$u_{blend} = \alpha u_{MPC} + (1 - \alpha) u_{LQR}, \quad \alpha \in [0, 1]. \quad (30)$$

This baseline is included because it is a common intuitive hybrid strategy. However, it does not guarantee obstacle avoidance, since the convex combination of a safe command and an unsafe command is not necessarily safe with respect to nonlinear obstacle constraints.

X. ROS2/GAZEBO SIMULATION SETUP

The proposed method is evaluated in ROS2/Gazebo using a TurtleBot3 differential-drive robot. The controller publishes velocity commands to the robot and receives odometry and obstacle information from the simulation environment.

A. LiDAR and Obstacle Representation

The controller itself is sensor-agnostic and only requires local obstacle geometry or distance information. In Gazebo, obstacles may be provided either from a known obstacle publisher or extracted from simulated LiDAR scans. For LiDAR-based experiments, the LaserScan data is converted into local obstacle points or circular obstacle approximations, which are then used by the risk predictor and adaptive MPC safety constraints.

B. Injected Actuation Mismatch

To evaluate adaptation, a command-distortion node is inserted between the controller and the robot. Given a commanded velocity

$$u_c = [v_c \quad \omega_c]^T, \quad (31)$$

the robot receives

$$u_a = [\lambda_v(k)v_c \quad \lambda_\omega(k)\omega_c]^T, \quad (32)$$

where $\lambda_v(k)$ and $\lambda_\omega(k)$ simulate motor degradation, battery drop, wheel slip, or calibration error. Example mismatch profiles include constant scaling, sinusoidal fluctuation, and step changes.

C. Sensor Noise

Gaussian noise is injected into odometry or LiDAR measurements:

$$\tilde{p}_x = p_x + \mathcal{N}(0, \sigma_p^2), \quad (33)$$

$$\tilde{p}_y = p_y + \mathcal{N}(0, \sigma_p^2), \quad (34)$$

$$\tilde{\theta} = \theta + \mathcal{N}(0, \sigma_\theta^2). \quad (35)$$

This tests whether the LMS adaptation and LQR tracking remain stable under imperfect sensing.

D. Dynamic Obstacles

In addition to static obstacles, dynamic obstacle scenarios are included. Each moving obstacle is modeled as

$$p_{obs,j}(i) = p_{obs,j}(0) + i\Delta t v_{obs,j}, \quad (36)$$

where $v_{obs,j}$ is the estimated obstacle velocity. The predictive rollout then checks future robot states against future obstacle positions.

XI. ALGORITHM

At every control step:

- 1) Measure current robot state x_k .
- 2) Extract local obstacle information from LiDAR, map, or obstacle publisher.
- 3) Update $\hat{\theta}_k$ using LMS with excitation thresholding.
- 4) Compute nominal command $u_{LQR,k}$.
- 5) Roll out the LQR command using the adapted model.
- 6) If the predicted rollout is safe, apply $u_{LQR,k}$.
- 7) If unsafe, solve the adaptive MPC safety filter.
- 8) If MPC succeeds within the time limit, apply the first MPC command.
- 9) If MPC fails, apply the backup safety controller.

XII. EVALUATION METRICS

The following metrics are reported.

A. Tracking Error

The trajectory tracking RMSE is

$$\text{RMSE} = \sqrt{\frac{1}{T} \sum_{k=1}^T ((p_{x,k} - p_{x,ref,k})^2 + (p_{y,k} - p_{y,ref,k})^2)}. \quad (37)$$

B. Minimum Obstacle Clearance

The minimum clearance is

$$d_{min} = \min_{k,j} (\|p_k - p_{obs,j}\|_2 - r_{obs,j}). \quad (38)$$

C. Collision Count

A collision is counted when

$$\|p_k - p_{obs,j}\|_2 < r_{obs,j} + r_{robot}. \quad (39)$$

D. Solver Time Profiling

The execution time of the control loop is recorded at every step:

$$t_{solve,k} = t_{end,k} - t_{start,k}. \quad (40)$$

The proposed method is expected to have low computation time during LQR-only operation and short spikes only when the adaptive MPC safety filter is activated.

E. Intervention Rate

The MPC intervention rate is

$$\rho_{MPC} = \frac{N_{MPC}}{N_{total}}, \quad (41)$$

where N_{MPC} is the number of timesteps where the safety filter overrides LQR.

F. Jerk and Smoothness

Control smoothness is measured using discrete jerk. For the linear command,

$$j_v(k) = \frac{v_k - 2v_{k-1} + v_{k-2}}{\Delta t^2}. \quad (42)$$

For the angular command,

$$j_\omega(k) = \frac{\omega_k - 2\omega_{k-1} + \omega_{k-2}}{\Delta t^2}. \quad (43)$$

The RMS jerk is reported as

$$J_{rms} = \sqrt{\frac{1}{T-2} \sum_{k=3}^T (j_v(k)^2 + j_\omega(k)^2)}. \quad (44)$$

G. Parameter Estimation Error

If ground-truth scaling parameters are known in simulation, the estimation error is

$$e_\vartheta = \|\hat{\vartheta}_k - \vartheta_{true,k}\|_2. \quad (45)$$

XIII. VISUALIZATION

Three main visualizations are used:

- 1) A top-down trajectory plot showing reference path, robot path, and obstacle positions.
- 2) An intervention map where green indicates LQR-only operation, yellow indicates adaptive MPC intervention, and red indicates backup safety activation.
- 3) A solver-time plot showing computational cost over time and highlighting MPC activation intervals.

XIV. EXPECTED RESULTS

The proposed method is expected to:

- reduce collision rate compared with LQR-only control,
- reduce average computation time compared with continuous adaptive MPC,
- provide better safety than blended LQR-MPC,
- maintain smoother commands than hard switching,
- adapt to actuation mismatch through LMS parameter estimation.

XV. CONCLUSION

This paper proposes a risk-triggered LMS-adaptive MPC safety filter for LQR-based differential-drive robot navigation. The method reframes hybrid LQR-MPC control as a safety-filtering problem rather than a switching or blending problem. By combining nominal LQR tracking, online actuation adaptation, predictive safety filtering, and backup safety control, the proposed architecture provides a practical and theoretically defensible approach for obstacle-aware TurtleBot3 navigation in ROS2/Gazebo.

REFERENCES

- [1] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed., Nob Hill Publishing, 2017.
- [2] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, 2017.
- [3] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, 2021.
- [4] J. Kohler, "Certainty-equivalent adaptive MPC for uncertain nonlinear systems," arXiv preprint, 2026.
- [5] Y. Wang et al., "Adaptive model predictive control for mobile robot trajectory tracking with localization fluctuation," *Sensors*, 2023.